

Tarea 2

Redes neuronales feedforward, verosimilitud y regularización

Table of contents

1. Red neuronal feedforward desde cero	3
2. Comparación con PyTorch	4
3. Verosimilitud para una variable objetivo positiva	4
4. Regularización explícita, implícita y heurística	5
5. Sensibilidad, ruido y sesgo-varianza	6
6. Preguntas de discusión	6
7. Red neuronal con salidas mixtas	7
Entregables	8

En esta tarea trabajaremos con redes neuronales *feedforward* o multilayer perceptrons (MLP). El objetivo es conectar tres ideas vistas en clases:

1. Una red neuronal como composición de transformaciones afines y funciones de activación.
2. El entrenamiento como minimización de una función de pérdida derivada desde máxima verosimilitud.
3. La evaluación de generalización mediante capacidad del modelo, sesgo-varianza y estrategias de regularización.

El problema se contextualiza en la predicción de demanda operacional para un sistema de bicicletas compartidas. Suponga que se quiere estimar el número esperado de viajes diarios en una estación a partir de variables de calendario, clima y operación. Por definición, la demanda real no puede ser negativa, pero un modelo de regresión estándar con pérdida cuadrática no impone esta restricción de manera explícita.

Trabajaremos con un conjunto de datos sintético. Puede usar el siguiente generador:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def generar_datos_demanda(n=1200, ruido=0.25, seed=42):
```

```

rng = np.random.default_rng(seed)

temperatura = rng.uniform(0, 35, size=n)
lluvia = rng.gamma(shape=1.5, scale=4.0, size=n)
hora_punta = rng.uniform(0, 1, size=n)
dia_laboral = rng.binomial(1, 0.72, size=n)
eventos = rng.poisson(0.25, size=n)

X = np.column_stack([
    temperatura,
    lluvia,
    hora_punta,
    dia_laboral,
    eventos,
])

# Funcion latente no lineal. La variable observada se genera como positiva.
g = (
    2.0
    + 0.045 * temperatura
    - 0.055 * lluvia
    + 1.1 * np.sin(np.pi * hora_punta)
    + 0.35 * dia_laboral
    + 0.22 * eventos
    - 0.0025 * temperatura * lluvia
)

y = np.exp(g + rng.normal(0, ruido, size=n))

columnas = ["temperatura", "lluvia", "hora_punta", "dia_laboral", "eventos"]
return pd.DataFrame(X, columns=columnas), y

X, y = generar_datos_demanda()

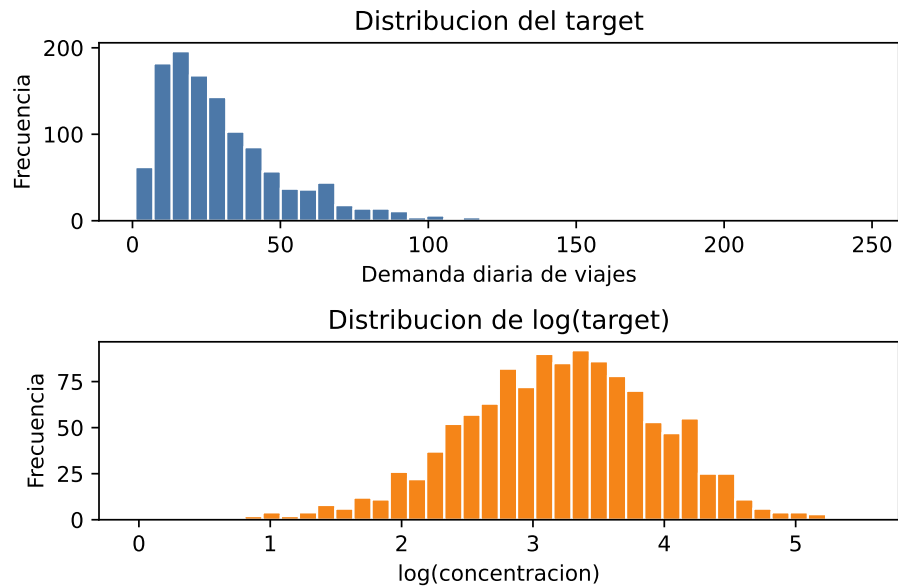
fig, axes = plt.subplots(2, 1, figsize=(6, 4))

axes[0].hist(y, bins=40, color="#4c78a8", edgecolor="white")
axes[0].set_title("Distribucion del target")
axes[0].set_xlabel("Demanda diaria de viajes")
axes[0].set_ylabel("Frecuencia")

axes[1].hist(np.log(y), bins=40, color="#f58518", edgecolor="white")
axes[1].set_title("Distribucion de log(target)")
axes[1].set_xlabel("log(concentracion)")
axes[1].set_ylabel("Frecuencia")

```

```
plt.tight_layout()
plt.show()
```



Antes de entrenar, divida los datos en conjuntos de entrenamiento, validación y prueba. Estandarice las variables de entrada **usando sólo estadísticas del conjunto de entrenamiento**. No use el conjunto de prueba para seleccionar arquitectura, hiperparámetros, número de épocas ni criterios de parada.

1. Red neuronal feedforward desde cero

Implemente una red neuronal *feedforward* para regresión usando únicamente librerías base como `numpy`, `pandas` y `matplotlib`. No use `torch`, `tensorflow`, `jax`, `sklearn.neural_network` ni diferenciación automática para esta parte.

Su implementación debe incluir:

- Una arquitectura completamente conectada con al menos una capa oculta.
- Funciones de activación no lineales, por ejemplo ReLU o `tanh`.
- Inicialización de pesos justificada, por ejemplo Xavier o He.
- *Forward pass*.
- Cálculo de la pérdida.
- *Backpropagation* implementado manualmente.
- Entrenamiento con descenso de gradiente por minibatches.
- Registro de la pérdida de entrenamiento y validación durante las épocas.

Comience entrenando el modelo con pérdida cuadrática media:

$$L(\phi) = \frac{1}{I} \sum_{i=1}^I (y_i - f[x_i, \phi])^2.$$

Reporte al menos tres configuraciones de arquitectura, variando profundidad y/o número de unidades ocultas. Para cada una, informe:

- Número de parámetros entrenables.
- Curvas de pérdida de entrenamiento y validación.
- Error en el conjunto de prueba usando MAE, RMSE y error relativo medio.
- Una figura de predicción versus valor real.
- Una breve discusión sobre subajuste, sobreajuste y capacidad del modelo.

2. Comparación con PyTorch

Implemente en PyTorch una red equivalente a la mejor arquitectura encontrada en la parte anterior. La comparación debe ser justa, es decir, use la misma partición de datos, el mismo preprocesamiento y una arquitectura comparable.

Compare la implementación desde cero con PyTorch en cuanto a:

- Desempeño predictivo en el conjunto de prueba.
- Velocidad de entrenamiento.
- Estabilidad de la optimización.
- Sensibilidad a la tasa de aprendizaje.
- Facilidad para cambiar arquitectura, función de pérdida y regularización.

Discuta posibles diferencias. Considere aspectos como inicialización, optimizadores, precisión numérica, tamaño de minibatch, criterios de parada y cálculo manual de gradientes.

3. Verosimilitud para una variable objetivo positiva

La demanda diaria es estrictamente positiva. Por lo tanto, una pérdida cuadrática con salida lineal puede ser inadecuada, ya que no representa explícitamente el dominio del target ni su posible asimetría.

Modifique la formulación probabilística del modelo para que la distribución condicional de la variable objetivo tenga soporte positivo. Debe elegir al menos una de las siguientes alternativas:

1. **Modelo log-normal:** suponga que $\log(y)$ sigue una distribución normal. La red puede predecir $\mu(x)$ y opcionalmente $\sigma(x)$ o una varianza global.
2. **Modelo gamma:** suponga que y sigue una distribución Gamma. La red debe predecir parámetros positivos usando una transformación como `softplus` o `exp`.
3. **Modelo normal truncado o salida positiva:** use una salida positiva para el modelo y derive claramente qué pérdida está minimizando.

Para la alternativa elegida:

- Escriba la verosimilitud $p(y_i | x_i, \phi)$.
- Derive la pérdida de log-verosimilitud negativa.
- Explique qué parámetros de la distribución son predichos por la red.
- Indique cómo garantiza que los parámetros que deben ser positivos efectivamente lo sean.
- Implemente la pérdida en su red *from scratch* o en PyTorch.
- Compare sus resultados contra el modelo con MSE.

Incluya una discusión sobre el efecto de esta modificación en las predicciones bajas. En particular: ¿disminuye la frecuencia de predicciones negativas o cercanas a cero?, ¿mejora el error relativo para estaciones con baja demanda?, ¿cambia la calibración de la incertidumbre si el modelo predice una varianza?

4. Regularización explícita, implícita y heurística

Diseñe experimentos para estudiar estrategias de regularización. Debe evaluar al menos cuatro de las siguientes:

- Regularización ℓ_2 sobre los pesos, sin penalizar sesgos.
- Regularización ℓ_1 o combinación ℓ_1 - ℓ_2 .
- *Early stopping* usando el conjunto de validación.
- *Dropout*.
- Reducción o aumento del tamaño de minibatch.
- Aumento de datos mediante perturbaciones controladas de las entradas.
- *Ensembling* de modelos entrenados con distintas semillas.
- Cambio de capacidad del modelo mediante profundidad o ancho de capas.

Para cada estrategia seleccionada, antes de entrenar el modelo escriba qué espera que ocurra y por qué. Después del experimento, compare esa expectativa con los resultados usando tablas y figuras. Por ejemplo: “esperamos que dropout aumente el error de entrenamiento, pero reduzca el error de validación”. Luego verifique si esto ocurrió observando las curvas de pérdida y las métricas en validación/prueba.

Responda:

1. ¿Qué estrategia redujo más la brecha entre pérdida de entrenamiento y validación?
2. ¿Qué estrategia mejoró más el desempeño en prueba?
3. ¿Qué estrategia empeoró el desempeño y por qué?
4. ¿Qué diferencias observa entre regularización explícita, como $\lambda g[\phi]$, y regularización implícita inducida por SGD?
5. ¿Cómo cambia el resultado al modificar el tamaño de minibatch manteniendo fija la arquitectura?
6. ¿En qué casos el *early stopping* se comporta de manera similar a una penalización ℓ_2 ?

7. ¿Existe evidencia de doble descenso o de una relación no monótona entre capacidad y error de prueba?
8. Si usa dropout, ¿cómo cambia el comportamiento cuando aumenta la probabilidad de apagar unidades?
9. Para la red implementada *from scratch*, compare las magnitudes de los pesos del modelo sin regularización y del modelo regularizado. Reporte al menos una norma global de los pesos, por ejemplo $\|\phi\|_2$ o $\|\phi\|_1$, y una visualización de la distribución de pesos por capa. ¿La regularización produjo pesos de menor magnitud? ¿El efecto fue igual en todas las capas?

5. Sensibilidad, ruido y sesgo-varianza

Repita una parte de sus experimentos bajo diferentes niveles de ruido en el generador de datos. Use al menos tres valores para el parámetro **ruido**, por ejemplo 0.10, 0.25 y 0.50.

Analice:

- Cómo cambia el error irreducible esperado.
- Cómo se modifica la brecha entre entrenamiento y validación.
- Qué arquitecturas son más sensibles al ruido.
- Qué estrategias de regularización se vuelven más importantes cuando el ruido aumenta.
- Si aumentar la cantidad de datos reduce la varianza observada entre distintas semillas.

Conecte su análisis con la descomposición conceptual del error en ruido, sesgo y varianza. No es necesario estimar formalmente los tres términos, pero sí debe argumentar cuál de ellos parece dominar en cada escenario.

6. Preguntas de discusión

Responda de manera breve y precisa:

1. ¿Por qué una red con ReLU implementa una función lineal por partes?
2. ¿Qué información debe guardarse durante el *forward pass* para poder ejecutar *backpropagation* eficientemente?
3. ¿Por qué una red suficientemente grande puede tener pérdida de entrenamiento casi cero y aun así generalizar mal?
4. ¿Qué rol cumple el conjunto de validación en la selección de hiperparámetros?
5. ¿Por qué no se debe usar el conjunto de prueba para decidir cuándo detener el entrenamiento?
6. ¿Qué diferencia práctica hay entre minimizar MSE y minimizar una NLL derivada de una distribución log-normal?
7. Si dos modelos tienen RMSE similar, pero uno nunca predice valores negativos, ¿cuál preferiría para este problema y por qué?

8. ¿Cómo se interpreta la regularización ℓ_2 desde una perspectiva probabilística MAP?
9. ¿Por qué usualmente se regularizan los pesos, pero no los sesgos?
10. ¿Qué costo computacional se tiene al usar un ensamble y cuándo se justificaría?

7. Red neuronal con salidas mixtas

En muchos problemas reales, una misma entrada debe alimentar varias predicciones de naturaleza distinta. Extienda el problema anterior para que la red tenga tres salidas:

1. **Salida continua:** una variable real que puede tomar valores positivos o negativos, por ejemplo el desvío de demanda respecto del promedio histórico de la estación.
2. **Salida continua positiva:** una variable estrictamente positiva, por ejemplo la demanda diaria esperada de viajes.
3. **Salida categórica:** una clase operacional, por ejemplo *baja*, *normal* o *alta* congestión del sistema.

Puede generar estos tres targets de manera sintética a partir de las mismas variables de entrada, o elegir una base de datos simple y definir targets razonables. En ambos casos debe explicar cómo se construyeron las salidas y qué representa cada una.

Para esta parte **no se exige implementación from scratch**. Implemente la red en PyTorch con un tronco compartido y tres cabezales de salida. Defina una función de costo total apropiada para este caso, por ejemplo:

$$L_{\text{total}} = \lambda_1 L_{\text{continua}} + \lambda_2 L_{\text{positiva}} + \lambda_3 L_{\text{categórica}},$$

donde puede usar MSE o MAE para la salida continua, una NLL log-normal/Gamma o una transformación positiva para la salida positiva, y entropía cruzada para la salida categórica.

Responda:

- ¿Qué función de pérdida eligió para cada salida y por qué?
- ¿Cómo garantiza que la salida positiva respete su dominio?
- ¿Cómo eligió o ajustó los pesos $\lambda_1, \lambda_2, \lambda_3$?
- ¿Qué métricas reporta para cada salida?
- ¿Observa conflicto entre tareas, por ejemplo mejora en una salida y empeoramiento en otra?
- ¿Qué ventajas y desventajas tiene usar un tronco compartido en lugar de tres modelos separados?

Entregables

Debe entregar un notebook reproducible que incluya lo siguiente:

- Código completo de la red neuronal desde cero.
- Código de la implementación equivalente en PyTorch.
- Derivación matemática de la pérdida positiva elegida.
- Tablas comparativas de resultados.
- Figuras de entrenamiento, validación y predicción.
- Análisis de regularización y sensibilidad al ruido.
- Implementación en PyTorch de la red con salidas mixtas: continua, positiva y categórica.
- Respuestas a las preguntas de discusión.
- Conclusiones sobre qué modelo recomendaría y bajo qué restricciones.

La evaluación considerará la corrección de la implementación, la claridad de las derivaciones, el diseño experimental, la calidad de las visualizaciones y la profundidad del análisis.